# Krkn-operator

Version 1.0.0, 2026-08-18

# Home

## Introduction

Course Title: Krkn-operator

Description: Course description FIXME

Duration: FIXME hours

## Topics covered in this course

- Introduction to Krkn-operator
- Installation
- Configuration and Management
- Lifecycle Management
- Hands-on Lab

## Prerequisites

This course assumes that you have the following prior experience:

- Course or experience…
- Course or experience…
- Course or experience…

# Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

# Introduction to Krkn-operator

## Introduction to Krkn-operator

The Krkn-operator is a Kubernetes-native platform designed to streamline Chaos Engineering by centralizing the deployment, configuration, and execution of experiments. It moves beyond manual scenario execution by providing an integrated web interface for comprehensive cluster and chaos orchestration.

Key features and capabilities include:

- **Centralized Orchestration:** Manage chaos experiments, user permissions, and cluster configurations from a single, unified web console.
- **Kubernetes-Native Architecture:** Built to operate directly within Kubernetes environments, leveraging native resources for seamless integration.
- **Chaos Management:** Simplifies the lifecycle of chaos engineering, from initial deployment to the execution and monitoring of complex experiments.
- **Operational Efficiency:** Eliminates the need for manual command-line execution by offering a robust management platform for chaos scenarios.

## Overview of Kubernetes-native Chaos Engineering

### Overview of Kubernetes-native Chaos Engineering

Kubernetes-native Chaos Engineering is the practice of proactively injecting controlled failures into a Kubernetes environment to identify weaknesses in system resiliency. By systematically disrupting components—such as killing pods, inducing network latency, or exhausting resources—you can observe how your applications and infrastructure react under stress.

The primary goal is to shift from reactive firefighting to proactive design, ensuring that your systems can gracefully recover from failures before they impact end-users.

#### The Role of Krkn-operator

In the context of the Kraken (Krkn) ecosystem, the **Krkn-operator** serves as the control plane for orchestrating these experiments within a Kubernetes cluster. Unlike manual script execution, the Krkn-operator provides a structured, automated way to lifecycle-manage chaos experiments.

##### Core Pillars of Krkn Chaos Engineering

- **Observability-Driven:** Integration with PromQL-based metrics allows you to measure the impact of chaos in real-time, providing a "Resiliency Score" for your applications.
- **Automation:** By moving away from manual `kubectl` commands, the operator allows for the definition of reusable, version-controlled chaos scenarios.
- **Safety and Control:** The architecture is designed to allow granular permission management, ensuring that chaos experiments are scoped strictly to intended targets.

- **Multi-Cluster Orchestration:** Chaos is rarely isolated to a single node. The Krkn-operator architecture supports executing experiments across multiple clusters, simulating real-world regional outages or large-scale control plane issues.

#### Why Kubernetes-Native?

Traditional chaos engineering tools often operate at the infrastructure level (e.g., shutting down virtual machines). Kubernetes-native chaos engineering, facilitated by the Krkn-operator, focuses on the orchestration layer:

1. **Abstraction of Complexity:** Through tools like `krknctl` and the operator-based web interface, users do not need to manage the underlying container runtime complexities.
2. **Kubernetes Resource Awareness:** Experiments understand Kubernetes primitives. For example, a scenario can target a specific `Deployment` or `StatefulSet` rather than just a random IP address, ensuring the experiment maintains semantic relevance to your application architecture.
3. **CI/CD Integration:** Because chaos scenarios are defined as reproducible configurations (or container images within the `krkn-hub`), they can be treated as first-class citizens in a CI/CD pipeline, allowing developers to test resiliency as part of the standard deployment process.

#### Architectural Components

- **Krkn-hub:** A repository of pre-built container images containing specific chaos scenarios.
- **Krkn-operator:** The controller that watches for Chaos Custom Resources (CRDs) within your cluster, triggering the execution of scenarios from the hub.
- **Chaos Studio:** A management layer that enables the design of complex workflows (serial or parallel execution) and monitors the health of the target cluster during the experiment.

By leveraging the Krkn-operator, teams can move from simply "knowing" their system is fragile to actively validating their auto-scaling policies, pod disruption budgets, and failover logic in a safe, controlled manner.

# Architecture and core components

## Architecture and core components

The Krkn-operator is designed as a Kubernetes-native controller, leveraging the Operator pattern to manage the lifecycle of chaos engineering experiments directly within the cluster. By utilizing custom resource definitions (CRDs) and a control loop, it provides a declarative way to orchestrate complex failure scenarios.

#### Architectural Overview

The architecture follows the standard Kubernetes controller pattern, ensuring that the desired state of chaos experiments is continuously reconciled with the actual state of the cluster.

```
graph TD
```

```
    A[User/Admin] -->|Apply CRD| B[Kubernetes API Server]
    B --> C[Krkn-Operator Controller]
    C -->|Orchestrates| D[Chaos Experiments]
    D -->|Injects Faults| E[Target Workloads/Nodes]
    C -->|Monitors| F[Cluster Health/Metrics]
    F -->|Feedback Loop| C
```

#### Core Components

The functionality of the Krkn-operator is powered by several internal building blocks:

- **Controller Manager:** The central brain of the operator. It watches for changes to Krkn-specific custom resources. When a new chaos scenario is defined, the controller triggers the execution logic.
- **Chaos Engine:** The execution engine responsible for the lifecycle of an experiment. It handles the injection of faults (e.g., CPU/Memory hogs, namespace deletions) and ensures that the system returns to its original state after the experiment duration expires.
- **Reconciliation Loop:** This component constantly monitors the health of the target environment. It ensures that if a component fails—whether due to infrastructure issues, application bugs, or dependencies—the system recognizes the failure immediately to avoid extended downtime.
- **Target Provider Interface:** An abstraction layer that allows the operator to interface with different infrastructure environments, such as standalone Kubernetes clusters or clusters managed via Advanced Cluster Management (ACM).

### Design Principles

To ensure reliability during chaos injection, the operator is built on these foundational principles:

| Principle | Description |
|---|---|
| | **High Availability** |
| The operator is designed for HA deployment, ensuring that the control plane for chaos engineering remains resilient even when the underlying infrastructure is under stress. | |
| **Minimal Recovery Time** | The operator logic prioritizes fast recovery. Once an experiment is concluded, the controller initiates recovery logic to restore normal operations, preventing "chaos residue." |
| | **Declarative Configuration** |

| | |
|---|---|
| Chaos scenarios are defined as Kubernetes objects. This allows teams to store, version, and audit chaos experiments using standard GitOps workflows. | |
| **Observability Integration** | The operator integrates with monitoring systems to verify that the system detects failures as soon as they are injected, validating the effectiveness of existing alerts and self-healing mechanisms. |

#### Component Responsibilities

The separation of concerns within the Krkn-operator architecture allows it to handle diverse failure modes—ranging from resource starvation (CPU/Memory Hogs) to systemic disruptions (Namespace Failures or Time Skewing). By offloading the execution to the controller, the operator ensures that even if the primary interface is disconnected, the scheduled experiments or recovery tasks remain active, maintaining control over the chaos.

# Use cases for chaos orchestration

## Use cases for chaos orchestration

Chaos orchestration is the systematic practice of injecting controlled failures into a system to verify its resilience against the inherent uncertainties of distributed environments. By leveraging the **Krkn-operator**, teams can move beyond manual testing and transition into automated, repeatable resilience validation.

#### Why Chaos Orchestration Matters

Distributed systems often fail due to assumptions that rarely hold true in production, such as the reliability of the network or the infinite availability of shared resources. Chaos orchestration addresses these by proactively challenging the system's design.

##### Key Use Cases

Through the Krkn-operator, you can implement the following strategic use cases:

| Use Case | Description |
|---|---|
| **Validation of High Availability (HA)** | Simulate the loss of individual nodes or pods to ensure that orchestration layers (like Kubernetes) correctly reschedule workloads and that service traffic reroutes without user-facing downtime. |
| **Dependency Resilience Testing** | Inject network latency or packet loss between microservices to evaluate how application code handles timeouts and circuit breaking. This ensures that a degradation in one component does not trigger a cascading failure across the entire system. |

| Use Case | Description |
| --- | --- |
| **Resource Exhaustion Profiling** | Artificially induce memory or CPU spikes on specific deployments. This helps verify that Horizontal Pod Autoscalers (HPA) trigger correctly and that the application behaves gracefully under load rather than crashing unexpectedly. |
| **Resiliency Score Benchmarking** | Use the Krkn-operator to measure your system against a "Steady State." By defining critical SLOs (e.g., API server latency), you can quantify how much your system's performance degrades during an experiment, providing a measurable **Resiliency Score**. |
| **Automated Compliance Verification** | Integrate chaos experiments into your CI/CD pipelines to ensure that every release maintains the required reliability standards before being promoted to production. |

#### Mapping Chaos to Business Goals

Chaos orchestration is not just about breaking things; it is about validating the **corrective** mechanisms built into your architecture.

- **Defining the Steady State:** Before orchestration begins, you must establish the "normal" behavioral baseline. Without this, you cannot objectively measure the impact of an experiment.
- **Analyzing Metrics:** During orchestration, the Krkn-operator allows you to monitor key KPIs. By comparing these against your established steady state, you can identify "blind spots" where monitoring or alerting might be insufficient.
- **Continuous Improvement:** Use the data gathered from orchestrated experiments to prioritize architectural changes. If an experiment proves a service is not resilient, the output provides the specific data needed to justify remediation efforts to stakeholders.

For complex environments, use **Chaos Studio**—a core feature of the Krkn ecosystem—to build visual workflows. This allows you to design scenarios that execute in series or parallel, enabling you to simulate multi-stage failure modes, such as a node failure occurring simultaneously with a network partition.

# Installation

## Installation

The `krkn-operator` is deployed on Kubernetes or OpenShift clusters primarily using Helm charts.

- **Prerequisites:** Ensure your environment meets the necessary cluster requirements before deployment.
- **Deployment Method:** Use Helm to install the operator, ensuring proper namespace management and creation to maintain an organized cluster environment.
- **Configuration:** Optionally attach configuration files via the File Management interface to customize your deployment parameters.

# Prerequisites for cluster deployment

## Prerequisites for cluster deployment

Before deploying the `krkn-operator` into your environment, it is essential to prepare your Kubernetes cluster to ensure compatibility, security, and resource availability. Chaos engineering experiments, by design, stress the underlying infrastructure; therefore, a well-prepared foundation is critical for both the stability of your platform and the validity of your chaos results.

### Cluster Environment Requirements

To ensure the `krkn-operator` functions correctly, your environment must meet the following criteria:

- **Kubernetes Version:** Ensure your cluster is running a supported version of Kubernetes. Refer to the official Krkn release notes for the specific minimum version required for the operator version you intend to deploy.
- **Permissions:** You must have `cluster-admin` privileges or appropriate RBAC permissions to create Namespaces, ClusterRoles, and Custom Resource Definitions (CRDs).
- **Connectivity:** The operator must have network reachability to the API server of the target clusters. If you are utilizing multi-cluster execution, ensure that the communication paths between the management cluster and target clusters are open.
- **Resource Sizing:**
- As noted in our best practices, applications often consume higher resource levels during reinitialization following a crash (e.g., replaying Write Ahead Logs in monitoring solutions like Prometheus).
- Ensure your worker nodes have sufficient headroom in terms of CPU and Memory to accommodate these spikes during recovery phases. Failing to do so can lead to resource contention that impacts both your application and system-critical pods.

#### Access and Configuration Prerequisites

Before the installation process begins, ensure you have the following components configured:

| Component | Requirement |
|---|---|
| **CLI Access** | `kubectl` or `oc` (for OpenShift environments) must be configured and authenticated against the target cluster. |
| **Helm** | The `helm` binary (version 3.x) is required for deploying the operator charts. |
| **Registry Access** | If your cluster resides in a restricted environment, ensure you have access to your private container registries. Configure image pull secrets in the target namespace beforehand to avoid `ImagePullBackOff` errors. |
| **Namespace Strategy** | Determine the dedicated namespace for the operator. While it can be installed cluster-wide, isolating the operator in a specific namespace (e.g., `krkn-chaos`) is recommended for security and observability. |

#### Best Practices for Chaos Readiness

Beyond technical requirements, consider these operational prerequisites:

- **Node Sizing:** Review your node allocation strategies. Chaos experiments should target specific components to understand their behavior under stress. If your nodes are tightly packed with production workloads, ensure that the "fast recovery logic" of your applications is tested prior to executing disruptive experiments.
- **Observability:** Confirm that your cluster has monitoring (e.g., Prometheus/Grafana) installed. The `krkn-operator` leverages PromQL-based metrics for the Resiliency Score feature. Without an active metrics pipeline, you will be unable to measure the impact of your experiments effectively.
- **Isolation:** Ensure that the target clusters are clearly identified. If using ACM/OCM integration for automatic discovery, ensure the credentials for those platforms are prepared and verified before attempting to register clusters within the operator interface.

Never execute chaos experiments on a production cluster without first validating the experiment workflows in a staging or development environment. Always monitor cluster health closely during the initial deployment phase to ensure the operator's own resource footprint does not interfere with existing workloads.

# Installing via Helm charts

## Installing via Helm charts

The `krkn-operator` is distributed as a Helm chart, facilitating standardized deployments across Kubernetes and OpenShift environments. By leveraging Helm, you can manage the operator lifecycle, perform version rollouts, and customize configurations through `values.yaml` files.

### Prerequisites

Before initiating the installation, ensure your environment meets the following requirements:

- **Kubernetes Cluster:** Version 1.19 or higher, or OpenShift 4.x.
- **Helm:** Version 3.0 or higher must be installed and configured in your local environment.
- **Cluster Access:** `cluster-admin` privileges (or sufficient permissions to create namespaces and CRDs) are required to install the operator.

### Installation Methods

The installation approach depends on how you intend to expose the operator's web interface for scenario orchestration.

| Environment | Recommended Access Method |
|---|---|
| **Kubernetes** | Gateway API (preferred) or Ingress |
| **OpenShift** | Routes (native) |

### Quick Start Deployment

For a standard installation using default settings, you can deploy the latest version (v0.4.5-beta) directly from the project's OCI registry. This command automatically creates the necessary namespace and deploys the operator components.

```
helm install krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system \
  --create-namespace
```

### Production Deployment with Custom Configuration

For production workloads, it is best practice to manage configuration via a custom `values.yaml` file. This allows you to define specific resource limits, replica counts, and exposure methods tailored to your cluster architecture.

1. Create a `values.yaml` file locally:

   ```
   # values.yaml example
   replicaCount: 1
   image:
     repository: quay.io/krkn-chaos/krkn-operator
     tag: 0.4.5-beta
   # Add environment-specific overrides here
   ```

2. Deploy the operator referencing your configuration file:

```
helm install krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system \
  --create-namespace \
  -f values.yaml
```

#### Verification

Once the Helm command completes, verify that the operator pods are running correctly:

```
kubectl get pods -n krkn-operator-system
```

Ensure the pods are in the `Running` state and that all containers have passed their readiness probes before attempting to configure chaos experiments.

# Namespace management and creation

## Namespace Management and Creation

In the context of the Krkn-operator, namespaces serve as the primary logical boundary for both the operator's control plane and the scope of chaos experiments. Proper namespace management is critical to ensuring that the Krkn-operator remains isolated, secure, and performant within your Kubernetes environment.

#### Overview of Namespace Strategy

The Krkn-operator requires a dedicated namespace to host its controller, web interface, and associated backend services. By isolating the operator, you prevent accidental interference with production workloads and ensure that Role-Based Access Control (RBAC) policies can be scoped effectively.

Key considerations for namespace management include: * **Isolation:** Keeping the operator separate from application workloads. * **Resource Quotas:** Applying resource limits to the operator namespace to ensure the chaos orchestration platform does not consume excessive cluster resources. * **RBAC Scoping:** Defining `RoleBindings` that allow the operator to manage resources in target namespaces without granting cluster-wide administrative privileges.

#### Managing Namespaces for Krkn-operator

To manage namespaces effectively, follow these best practices for setup and configuration:

##### 1. Creating the Operator Namespace

Before deploying the Krkn-operator, you must create a dedicated namespace. This acts as the home for the operator's pods.

```
kubectl create namespace krkn-operator-system
```

#### 2. Labeling for Security and Integration

If your cluster employs security policies (such as Pod Security Admission), ensure the namespace is labeled correctly to allow the operator pods to run with the necessary privileges.

```
kubectl label namespace krkn-operator-system pod-
security.kubernetes.io/enforce=privileged
```

### Hands-on Lab: Namespace Provisioning

In this activity, you will prepare your cluster by creating the required namespace for the Krkn-operator.

#### Prerequisites

- A running Kubernetes cluster.
- `kubectl` configured with cluster-admin privileges.

#### Step 1: Create the Operator Namespace

Execute the following command to create the workspace for the Krkn-operator:

```
kubectl create namespace krkn-operator
```

#### Step 2: Verify Namespace Readiness

Ensure the namespace exists and is ready to accept resource deployments:

```
kubectl get namespace krkn-operator
```

#### Step 3: Verify Permissions

As per the permission model, ensure that your administrative user has access to this namespace. If you are integrating this with the Krkn-operator's web interface later, you will need to map this namespace to a specific user group.

If you intend to run "Namespace Failures" (as defined in our documentation context), be aware that the Krkn-operator will require `delete` permissions on the target namespaces you designate for experimentation. Ensure your `ClusterRole` definitions reflect these requirements.

#### Step 4: Cleanup

If you need to remove the operator and clean up the namespace, use the following command:

```
kubectl delete namespace krkn-operator
```

Deleting a namespace is destructive and will remove all custom resource definitions (CRDs), chaos scenario configurations, and logs stored within that namespace. Always back up your configuration files before proceeding with deletion.

# Configuration and Management

## Configuration and Management

The Configuration and Management module provides the tools required to govern user access, infrastructure settings, and chaos experiment execution within the Krkn-operator platform.

- **User and Access Control:** Administrators manage users and groups, assigning permissions to ensure secure access to clusters and operational features.
- **Infrastructure Setup:** Configure core platform components, including target providers, private registries, and cluster registration.
- **Chaos Experimentation:** Define and manage reusable configuration files for chaos experiments and orchestrate complex scenarios directly through the web interface.
- **Resource Administration:** Utilize file management capabilities to organize text-based configurations for consistent use across Chaos Studio workflows.

# Managing users and permissions

### Managing Users and Permissions

The Krkn-operator utilizes a robust, group-based access control model to ensure secure multi-tenant chaos engineering. By decoupling permissions from individual users and associating them with functional groups, administrators can maintain a scalable security posture across multiple Kubernetes clusters.

#### The Permission Model

The security architecture is built on a hierarchical structure where the **Group** acts as the primary container for access policies. Every user within the platform must be assigned to at least one group; a user without a group assignment will have no visibility or operational access to the platform.

##### Key Components

- **Groups:** Logical entities created by Administrators that define the scope of interaction. A group configuration specifies:
- **Cluster Accessibility:** Which target clusters are available to members.
- **Permissions:** The granular operations allowed (View, Run, Delete).
- **Registry Visibility:** Which image registries can be accessed by the chaos experiments.
- **Users:** Individuals who inherit all permissions associated with their assigned group.

#### Role Definitions

Permissions are tailored to specific personas within your organization:

| Role | Responsibilities |
| --- | --- |
| **Administrator** | Manages the infrastructure, registers target clusters, creates groups, maps users to groups, and configures private registries and target providers (such as ACM). |
| **User** | Performs operational tasks based on group membership. This includes managing experiment jobs, accessing cluster terminals, executing chaos scenarios, and designing workflows in Chaos Studio. |

#### Operational Capabilities

The scope of a user's activity is determined by the permissions granted to their group:

- **View and Manage:** Ability to observe and interact with existing chaos jobs (requires View/Delete permissions).
- **Terminal Access:** Ability to open a shell into a target cluster (requires Run permission).
- **Scenario Execution:** Ability to trigger chaos experiments on assigned clusters (requires Run permission).
- **Chaos Studio:** Ability to design, modify, and save complex chaos workflows (requires Run permission).
- **File Management:** Ability to upload and manage files, scoped strictly by the group's visibility settings.

#### Admin Configuration Workflow

To manage the environment effectively, administrators follow this standard lifecycle:

1. **Define the Group:** Create a group and designate the specific target clusters that the group will interact with.
2. **Assign Permissions:** Select the appropriate permission level (View, Run, Delete) to define the group's capability on the assigned clusters.
3. **Assign Users:** Add users to the group. Upon assignment, the user immediately inherits the group's security context.

To ensure the principle of least privilege, audit your group assignments regularly. A user's access is strictly defined by their group—if a user needs access to a new cluster, add that cluster to their existing group or assign them to an additional group with the required access.

#### Troubleshooting Permission Issues

If a user reports that they cannot see a cluster or execute a command, verify the following:

- **Group Membership:** Ensure the user is explicitly assigned to a group.
- **Group Policies:** Check if the target cluster is correctly mapped to the user's group.
- **Permission Level:** Confirm that the user has the 'Run' permission, as this is a prerequisite for

executing scenarios and using terminal features.

# Configuring chaos experiments

## Configuring Chaos Experiments

Chaos experiments are the fundamental unit of work within the `Krkn-operator`. They define the "fault" or "stress" to be injected into your Kubernetes environment. Configuring these experiments requires a clear understanding of the experiment definition, the target scope, and the desired steady-state hypothesis.

### Anatomy of a Chaos Experiment

In the context of the `Krkn-operator`, a chaos experiment is typically managed through custom resources (CRs) that define the parameters of the disruption. When configuring these, focus on three primary pillars:

1. **Targeting:** Identifying the specific resources (Pods, Nodes, Deployments, or Network policies) that will be subjected to the fault.
2. **Disruption Parameters:** Defining the type of stress (e.g., CPU/Memory hogging, network latency, pod kills) and its intensity.
3. **Guardrails:** Defining constraints to ensure the chaos does not exceed the acceptable impact boundaries, potentially using the Krkn signaling feature to pause or stop if health checks fail.

### Best Practices for Configuration

When drafting your experiment configurations, adhere to the following principles:

- **Define the Scope:** Avoid broad, cluster-wide impacts unless explicitly testing global resiliency. Use label selectors to pin experiments to specific subsets of your infrastructure.
- **Observe, Don't Just Break:** Always pair your experiment with observability tools. A chaos experiment without telemetry is simply a system outage.
- **Steady-State Hypothesis:** Before configuring the experiment, define what "normal" looks like. The experiment configuration should be designed to validate that the system returns to this state after the disruption ceases.
- **Environment Sensitivity:** As noted in our testing guide, experiments configured for staging may need different thresholds than those intended for production.

### Hands-on Lab: Configuring a Basic Pod-Kill Experiment

This activity guides you through creating a simple chaos experiment configuration to terminate pods within a specific namespace.

#### Prerequisites

- A running Kubernetes cluster.
- `Krkn-operator` installed and running.

- A target application deployed in a namespace (e.g., `my-app`).

#### Step 1: Create the Experiment Manifest

Create a file named `pod-kill-experiment.yaml`. This configuration instructs the operator to target a deployment by its label.

```
apiVersion: krkn.redhat.com/v1beta1
kind: ChaosExperiment
metadata:
  name: pod-killer-demo
  namespace: krkn-installer
spec:
  experiment:
    type: "pod-kill"
    labelSelector: "app=my-web-service"
    namespace: "my-app"
    duration: "60s"
    count: 2 # Kill 2 pods
```

#### Step 2: Apply the Configuration

Apply the manifest to your cluster using `kubectl`:

```
kubectl apply -f pod-kill-experiment.yaml
```

#### Step 3: Verify Execution

Monitor the status of your experiment using the custom resource status field:

```
kubectl get chaosexperiment pod-killer-demo -n krkn-installer -o yaml
```

Observe the `status` block. You should see the progression from `Pending` to `Running`, and finally `Completed`.

#### Step 4: Validate Impact

Check your target namespace to verify the pods were restarted:

```
kubectl get pods -n my-app
# Look for the 'Age' column to confirm pod recreation
```

If the experiment does not trigger as expected, check the `krkn-operator` controller logs. Use `kubectl logs -l name=krkn-operator -n krkn-installer` to identify potential permission issues or invalid label selectors.

# Orchestrating scenarios via the web interface

## Orchestrating scenarios via the web interface

The Krkn-operator web interface provides a centralized control plane for chaos orchestration, moving beyond command-line execution to a visual, intuitive workflow management system. This interface allows operators to configure, execute, and monitor resilience tests across Kubernetes clusters seamlessly.

### Core Interface Components

The web interface is designed to streamline the lifecycle of a chaos experiment through the following key modules:

- **Run Scenarios:** The primary gateway for executing single chaos scenarios. It allows for quick testing of specific failure injections without the overhead of complex workflow design.
- **Chaos Studio:** A drag-and-drop environment for designing sophisticated chaos experiments. It supports both serial and parallel execution, enabling the construction of complex failure graphs to simulate real-world production outages.
- **Jobs:** A real-time monitoring dashboard where you can inspect scenario execution progress, logs, and final results.
- **File Management:** A built-in repository for uploading and managing custom configuration files (YAML/JSON) required for specific experiments.
- **Cluster Terminal:** A read-only utility that allows you to explore target clusters, providing visibility into the state of your infrastructure during or after chaos execution.

### Configuring Scenario Parameters

Before launching any scenario, you must define the operational parameters. These are categorized into three distinct levels:

1. **Mandatory:** Parameters required for the scenario to function (e.g., target namespace, duration). The operator will flag these as "must-configure" before the launch button is enabled.
2. **Optional:** Settings that provide fine-grained control over the experiment, such as specific label selectors, resource filters, or timing delays.
3. **Global:** Framework-level settings that govern the overall environment. This includes integration points like Prometheus for metrics, Elasticsearch for logging, and Cerberus for status monitoring.

### Executing and Monitoring

Once the parameters are defined, orchestration follows a simple two-step process:

1. **Launch:** Initiate the scenario or workflow via the web interface.
2. **Monitor:** You will be automatically redirected to the **Jobs** page. Here, you can watch the scenario’s execution in real-time, inspect resource transitions, and verify if the resiliency score

meets your organizational thresholds.

### Hands-on Lab: Executing a Scenario

In this exercise, you will trigger a sample chaos scenario using the web interface.

#### Prerequisites

- Krkn-operator successfully deployed in your cluster.
- Access to the web interface URL.

#### Step 1: Accessing the Run Scenarios Module

1. Navigate to the Krkn-operator dashboard.
2. Select **Run Scenarios** from the sidebar.
3. Select a pre-loaded scenario (e.g., `pod-kill`) from the library.

#### Step 2: Configuring Parameters

1. In the configuration panel, ensure all **Mandatory** fields are filled.
2. Under the **Optional** tab, add a `label selector` to target only the `app=frontend` pods.
3. Verify that the **Global** settings are connected to your monitoring stack.

#### Step 3: Launch and Observe

1. Click the **Run** button.
2. Immediately observe the **Jobs** page.
3. Watch as the status transitions from `Pending` to `Running` and finally to `Completed`.
4. Click on the job name to view the detailed logs and the resiliency output.

If a job fails, use the **Cluster Terminal** to inspect the target resource logs to ensure the chaos injection reached the intended pod or node.

# Lifecycle Management

## Lifecycle Management

Lifecycle management ensures the ongoing reliability and operational integrity of the Krkn-operator within a Kubernetes environment. Key activities include:

- **Upgrades:** Performing seamless operator updates to incorporate new features and performance improvements.
- **Version Control:** Managing releases and compatibility to ensure consistency across chaos experiment environments.
- **Maintenance:** Monitoring operator health to ensure stable execution of chaos scenarios and continued adherence to defined service level indicators.

## Performing operator upgrades

### Performing operator upgrades

Upgrading the `krkn-operator` is a critical administrative task to ensure your chaos engineering environment benefits from the latest features, security patches, and experiment definitions. As a Kubernetes-native tool, the operator lifecycle is managed primarily through Helm, allowing for seamless updates within your cluster.

#### Prerequisites for Upgrades

Before performing an upgrade, ensure the following conditions are met: * **Helm CLI:** Ensure you have the latest version of Helm installed locally. * **Cluster Access:** You must have `cluster-admin` or appropriate namespace-level permissions to modify the operator resources. * **Backup:** While the operator is stateless, ensure your custom chaos scenarios and experiment definitions are backed up or stored in version control (e.g., Git). * **Network Connectivity:** Ensure your cluster can reach the container registry at `quay.io` to pull the updated chart versions.

#### Performing the Upgrade

The `krkn-operator` is distributed as an OCI-compliant Helm chart. To upgrade the operator, you will point your local Helm client to the OCI registry and specify the desired version.

##### Step-by-Step Upgrade Procedure

1. **Verify Current Installation:** Check the currently installed version to confirm the release name and namespace: [source,bash] ---- helm list -n krkn-operator-system ----
2. **Execute the Upgrade Command:** Use the `helm upgrade` command to pull the latest version from the OCI registry. Replace `0.4.5-beta` with your target version: [source,bash] ---- helm upgrade krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \ --version 0.4.5-beta \ --namespace krkn-operator-system ----
3. **Validate the Upgrade:** After the command completes, verify that the operator pods have

restarted and are running the new image: [source,bash] ---- kubectl get pods -n krkn-operator-system ----

#### Troubleshooting Upgrade Issues

If the upgrade process encounters issues, follow these diagnostic steps:

- **Check Rollout Status:** If the operator fails to start, check the logs of the manager pod: [source,bash] ---- kubectl logs -l control-plane=controller-manager -n krkn-operator-system ----
- **Inspect Helm History:** If you need to revert to a previous working state, use the rollback feature: [source,bash] ---- helm rollback krkn-operator <revision_number> -n krkn-operator-system ----
- **Permission Errors:** If you receive "Access Denied" errors, verify that your service account or kubeconfig has the necessary permissions to update deployments and cluster roles within the `krkn-operator-system` namespace.

#### Best Practices

- **Version Pinning:** Always specify a `--version` flag to avoid accidental updates to unstable or incompatible releases.
- **Read Release Notes:** Before upgrading, consult the official release notes for any breaking changes in the Custom Resource Definitions (CRDs) or configuration schema.
- **Testing:** Perform upgrades in a non-production staging cluster before applying them to your production chaos orchestration environment.

# Handling versioning and releases

## Handling versioning and releases

As the `krkn-operator` evolves to support new Kubernetes primitives and advanced chaos scenarios, maintaining a clear versioning strategy is essential for cluster stability. The operator follows standard software lifecycle practices to ensure that updates do not inadvertently trigger destructive chaos events or interrupt observability pipelines.

#### Understanding Versioning Strategy

The `krkn-operator` follows [Semantic Versioning (SemVer)](https://semver.org/), which uses the format `MAJOR.MINOR.PATCH` (e.g., `1.2.4`). This structure allows users to anticipate the impact of an upgrade:

- **MAJOR version (X.0.0):** Indicates breaking changes. This often involves changes to the Custom Resource Definitions (CRDs) or architectural shifts in how the operator interacts with the `krkn-lib`. Upgrading across major versions may require manual intervention or reconfiguration of existing chaos scenarios.
- **MINOR version (x.Y.0):** Introduces new features, such as new chaos scenarios or expanded support for cloud-provider-specific disruptions, while maintaining backward compatibility.

- **PATCH version (x.y.Z):** Focuses on bug fixes, performance optimizations for `krkn-lib` integrations, or security patches. These are generally safe to apply in production environments.

#### Release Lifecycle

Releases for the `krkn-operator` are managed through the GitHub repository ecosystem. When a new release is cut, the following artifacts are updated:

1. **Helm Charts:** The repository's Helm chart version is incremented to match the operator image tag.
2. **Container Images:** New images are pushed to the container registry, tagged with the specific release version and a `latest` tag (though pinning to specific versions is recommended for production).
3. **Changelog:** Each release includes a detailed changelog highlighting new features, deprecated items, and fixed issues.

#### Best Practices for Upgrades

Before performing an upgrade in a production-grade cluster, we recommend the following workflow:

- **Review the Changelog:** Always check the release notes for "Breaking Changes." If you are using custom CRDs, ensure they remain compatible with the new operator version.
- **Use a Staging Environment:** Test the upgrade in a non-production cluster where `krkn-demos` can be run to verify that chaos scenarios execute as expected.
- **Verify CRD Compatibility:** Since the operator relies on Kubernetes-native objects, verify if the new release requires a `kubectl apply` of new or updated CRD manifests before updating the Helm deployment.
- **Monitor Operator Health:** Following an upgrade, observe the operator logs for initialization errors. Ensure the `krkn-lib` dependencies are correctly initialized and the operator is able to communicate with the Kubernetes API server.

#### Rollback Strategy

In the event that a new version introduces unexpected behavior, the operator can be reverted using Helm:

```
# List history of releases
helm history krkn-operator -n krkn-namespace

# Rollback to the previous stable release
helm rollback krkn-operator [REVISION_NUMBER] -n krkn-namespace
```

By maintaining strict version control and following the release lifecycle, teams can safely integrate Chaos Engineering into their continuous delivery pipelines while ensuring the `krkn-operator` remains a stable and predictable component of the infrastructure stack.

# Maintaining operator health

## Maintaining Operator Health

Maintaining the health of the `krkn-operator` is critical to ensuring that chaos experiments are executed reliably without compromising the stability of the management plane. A healthy operator ensures that chaos scenarios are orchestrated correctly, resources are cleaned up post-experiment, and the system remains observable.

### Health Monitoring Best Practices

To ensure the `krkn-operator` functions within expected parameters, consider the following maintenance strategies:

- **Resource Monitoring:** Monitor the memory and CPU utilization of the `krkn-controller-manager` pods. Excessive resource consumption may indicate memory leaks or an unoptimized reconciliation loop.
- **Log Analysis:** Regularly inspect operator logs for errors related to API connectivity or permission denials. Using `kubectl logs -n krkn-namespace deployment/krkn-controller-manager` is standard practice.
- **Condition Tracking:** Leverage Kubernetes status conditions. Ensure the operator reports `Ready` and `Available` statuses. If the operator enters a `CrashLoopBackOff`, investigate the configuration map for syntax errors or invalid experiment parameters.
- **Event Auditing:** Watch for Kubernetes events within the namespace where the operator is deployed to catch early warnings regarding resource quota exhaustion or RBAC failures.

### Ensuring System Stability

The stability of the operator is intrinsically linked to the underlying cluster health. Apply the following administration standards:

**Machine Health Checks:** Always deploy machine health checks with appropriate conditions to remediate unhealthy nodes. This ensures that even if a chaos experiment targets infrastructure, the cluster maintains enough capacity to run without total downtime, preventing the operator from struggling to reconcile state on degraded hardware.

#### Proactive Maintenance Steps

1. **Reconciliation Audits:** Periodically check the number of active chaos resources. An accumulation of orphaned `ChaosExperiment` custom resources (CRs) can bloat the etcd database and slow down the controller reconciliation loop.
2. **RBAC Verification:** If you modify service accounts or cluster roles, ensure the operator retains the necessary permissions to interact with target resources. Use `kubectl auth can-i` to verify that the operator's service account has the required verb access (e.g., `list`, `patch`, `delete`) to the target namespaces.

3. **Dependency Updates:** Periodically verify that the underlying Kubernetes API versions supported by the operator are compatible with your current cluster version.

#### Troubleshooting Health Issues

If the operator exhibits unexpected behavior, follow this diagnostic workflow:

1. **Check Pod Status:**

```
kubectl get pods -n krkn-operator-system
```

2. **Inspect Operator Logs:** Look for stack traces or connection timeouts.

```
kubectl logs -n krkn-operator-system -l control-plane=controller-manager --tail=100
```

3. **Verify Resource Limits:** Ensure that the `LimitRanges` and `ResourceQuotas` in the namespace do not restrict the operator from spawning the necessary pods to execute chaos experiments.

By integrating these maintenance tasks into your standard platform administration routine, you ensure that the `krkn-operator` remains a resilient tool for validating your service level indicators (SLIs) and key performance indicators (KPIs) under duress.

# Hands-on Lab

## Hands-on Lab

The hands-on lab provides a practical environment to apply Krkn-operator concepts through real-world scenarios. It focuses on the following key activities:

- **Deployment:** Installing the Krkn-operator within a Kubernetes cluster to establish the testing environment.
- **Experimentation:** Executing sample chaos experiments to observe system behavior and validate resilience.
- **Maintenance:** Performing operator upgrades to manage versioning and ensure the platform remains up-to-date with the latest features.

# Deploying the Krkn-operator in a Kubernetes cluster

## Deploying the Krkn-operator in a Kubernetes cluster

This section provides the procedure to deploy the Krkn-operator within your Kubernetes environment. The Krkn-operator acts as a centralized control plane for managing your chaos engineering experiments, allowing you to orchestrate scenarios through a unified web interface rather than managing manual executions.

### Prerequisites

Before deploying the Krkn-operator, ensure your environment meets the following requirements:

- **Kubernetes Cluster:** A functional cluster (Kubernetes or OpenShift).
- **Helm:** Ensure Helm 3.x is installed on your local machine to manage the deployment.
- **External Access:** Decide on your access strategy. For Kubernetes environments, it is recommended to use the **Gateway API** or **Ingress** to expose the web interface.
- **Permissions:** You must have `cluster-admin` privileges to create namespaces and manage Custom Resource Definitions (CRDs) required by the operator.

### Installation Procedure

The Krkn-operator is distributed via OCI-compliant Helm charts. Use the following steps to deploy the operator into your cluster.

#### Step 1: Create the Namespace

It is best practice to isolate the operator in its own namespace to maintain cluster hygiene.

```
kubectl create namespace krkn-operator-system
```

#### Step 2: Prepare the Configuration

While you can use default settings, a production-grade deployment typically requires a `values.yaml` file to define your specific ingress controller settings and persistent storage requirements.

Create a file named `values.yaml` and configure your parameters. For example, to enable OCM/ACM integration for automatic cluster discovery, include the following:

```
acm:
  enabled: true
# Add additional configuration for ingress or resource limits here
```

#### Step 3: Deploy via Helm

Execute the following command to install the Krkn-operator. This command pulls the chart directly from the official Krkn repository.

```
helm install krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system \
  -f values.yaml
```

If you are not using a `values.yaml` file, you can pass parameters directly via the command line. For instance, to enable ACM integration without a file: `--set acm.enabled=true`

### Verification

Once the Helm installation completes, verify that the operator components are running correctly by checking the status of the pods in the `krkn-operator-system` namespace:

```
kubectl get pods -n krkn-operator-system
```

Ensure all pods are in the `Running` or `Completed` state. If the pods are failing to start, inspect the logs for configuration errors:

```
kubectl logs -l app.kubernetes.io/name=krkn-operator -n krkn-operator-system
```

### Post-Deployment: Accessing the Interface

After the pods are stable, retrieve the address for the web interface based on the Ingress or Gateway API configuration provided in your `values.yaml`. You can then navigate to the provided URL to begin managing your chaos experiments.

# Executing a sample chaos experiment

## Executing a sample chaos experiment

This section guides you through the execution of a baseline chaos experiment using the `Krkn-operator`. We will trigger a pod-failure scenario to observe how your application and cluster infrastructure respond to unexpected disruptions.

### Prerequisites

Before executing the experiment, ensure: * The `Krkn-operator` is successfully deployed in your cluster. * You have access to the Kubernetes namespace where the target application resides. * The `krkn` CLI tool or the operator's Custom Resource Definition (CRD) interface is configured.

### Understanding the Experiment Workflow

Chaos experiments in the `Krkn-operator` are defined as Kubernetes Custom Resources (CRs). When you apply a chaos CR, the operator reconciles the state, coordinates the execution of the fault, and monitors the target resources.

### Hands-on Lab: Executing a Pod-Failure Experiment

In this exercise, we will induce a controlled pod-failure within a specific deployment to test the resilience of your service.

#### Step 1: Create the Chaos Experiment Manifest

Create a file named `pod-failure-experiment.yaml`. This manifest instructs the operator to terminate pods matching a specific label.

```
apiVersion: krkn.redhat.com/v1beta1
kind: ChaosExperiment
metadata:
  name: sample-pod-failure
  namespace: krkn-operator-system
spec:
  experiment_type: "pod-failure"
  target_namespace: "my-app-namespace"
  label_selector: "app=frontend"
  duration: "60s"
  instances: 1
```

#### Step 2: Trigger the Experiment

Apply the manifest to your cluster to initiate the chaos orchestration.

```
kubectl apply -f pod-failure-experiment.yaml
```

#### Step 3: Monitor Execution

Use the following commands to observe the operator's progress and the state of your target application:

1. **Check Operator logs:**

```
kubectl logs -f deployment/krkn-operator-controller-manager -n krkn-operator-system
```

2. **Observe the target pods:**

```
kubectl get pods -n my-app-namespace -w
```

#### Step 4: Analyze Resilience

Once the duration defined in the YAML has elapsed, the operator will stop the experiment. Review the results:

- **Observation:** Did the application deployment automatically spin up a new pod?
- **Monitoring:** Check your observability dashboard (e.g., Prometheus/Grafana) to see if latency spiked or if health checks failed during the pod termination.
- **Learning:** Remember that the goal is not to "pass" or "fail" the test, but to understand if the system recovers gracefully without manual intervention.

If you have configured the Kraken **Signaling feature**, you can use it here to pause other performance or scalability tests running in the background until the pod-failure experiment completes, ensuring that you isolate the chaos impact from other telemetry data.

#### Troubleshooting

If the experiment fails to start: * Verify that the `target_namespace` and `label_selector` match active resources in your cluster. * Ensure the service account used by the `Krkn-operator` has sufficient RBAC permissions to delete pods in the target namespace. * Check the status of the `ChaosExperiment` CR: `kubectl describe chaosexperiment sample-pod-failure -n krkn-operator-system`.

# Upgrading the operator to a newer version

## Upgrading the Krkn-operator

As your chaos engineering requirements evolve, keeping the `krkn-operator` up to date ensures you have access to the latest chaos scenarios, performance improvements, and security patches. This section guides you through the process of upgrading the operator within your Kubernetes environment.

### Prerequisites

- **Helm CLI**: Ensure you have the latest version of Helm installed.
- **Cluster Access**: You must have `cluster-admin` or appropriate permissions to modify resources within the `krkn-operator-system` namespace.
- **Version Verification**: Identify the target version you intend to upgrade to by checking the [Krkn-chaos repository](https://github.com/krkn-chaos/krkn-operator) or the OCI registry.

### Upgrade Procedure

Upgrading the `krkn-operator` is a straightforward process using the Helm CLI. Since the operator is distributed via OCI-compliant charts, you can perform an in-place upgrade without removing the existing installation.

Before performing an upgrade in a production-like environment, it is recommended to verify the release notes of the target version to identify any potential breaking changes in CRD (Custom Resource Definition) schemas or configuration parameters.

#### Step-by-Step Upgrade

1. **Update your local Helm repositories** (if applicable) to ensure you are referencing the latest metadata.
2. **Execute the upgrade command** using the `helm upgrade` directive. Ensure you point to the same namespace used during the initial installation.

```
helm upgrade krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system
```

- `--version`: Specifies the target release version.
- `--namespace`: Must match the namespace where the operator is currently deployed.

### Verifying the Upgrade

After executing the upgrade command, verify that the operator pods have rolled out successfully and are running the new image version.

1. **Check the status of the release:**

   ```
   helm status krkn-operator -n krkn-operator-system
   ```

2. **Monitor the operator pod rollout:**

   ```
   kubectl rollout status deployment/krkn-operator-controller-manager -n krkn-
   ```

```
operator-system
```

**Troubleshooting Common Issues**

- **Namespace Mismatch**: If you receive a "release not found" error, verify that the namespace provided in the `upgrade` command matches the namespace where the operator was originally installed.
- **CRD Conflicts**: If the upgrade fails due to CRD changes, ensure that you have the necessary cluster-level permissions to update Kubernetes custom resources.
- **Image Pull Errors**: If the pods are stuck in `ImagePullBackOff`, ensure your cluster has network connectivity to `quay.io` and that the OCI registry remains accessible.

# Appendix

FIXME: Content for Appendix…

## Resources

Download Course PDF